DSM 5007 DENETİMLİ İSTATİSTİKSEL ÖĞRENME FİNAL ÖDEVİ

Tarık Tuna Taşaltı

2025/01/26

İçindekiler

1 Veri Ön İşleme Aşamaları **2**
1.1 Veri Çarpıklığının İncelenmesi . . . 2
1.2 Box-cox Dönüşümü . . . 3
1.3 Veri Görselleştirmesi . . . 4
1.4 Verinin Eğitim - Test Şeklinde Bölünmesi . . . 5
1.5 Box-Cox Lambda Değerlerinin Eğitim Verisi Üzerinden Bulunması . . . 5
1.6 Box-Cox Dönüşümünün Test Verisine Uygulanması . . . 6

2 Regresyon Modellerinin Geliştirilmesi **7**
2.1 Regresyon Metriklerinin Tanımlanması . . . 7
2.2 Lineer Regresyon . . . 7
2.3 Regresyon Ağacı . . . 9
2.4 Bagging . . . 11
2.5 Random Forest . . . 11
2.6 Genel Değerlendirme . . . 12

3 Sınıflandırma Modelleri **14**
3.1 Sınıflandırma İçin Eşik Değerlerinin Belirlenmesi . . . 14
3.2 Sınıflandırma Metriklerinin Tanımlanması . . . 14
3.3 Sınıflandırma Ağacı . . . 14
3.4 Bagging . . . 16
3.5 Random Forest . . . 16
3.6 Lojistik Regresyon . . . 17

3.7 LDA (Linear Discriminant Analysis) 18
3.8 QDA (Quadratic Discriminant Analysis) 21
3.9 ROC - AUC 22
3.10 Sınıflandırma Sonuçlarının Değerlendirilmesi 24

1 Veri Ön İşleme Aşamaları

Csv dosyası okunmuştur. summary fonksiyonu kullanılarak NA değer olmadığı saptanmıştır. Overall.rank değişkeni çıkarılmıştır. Country.or.region her satır için unique değerler barındırdığı için satır ismi olarak tanımlanmıştır ve kolon veri setinden çıkarılmıştır.

1.1 Veri Çarpıklığının İncelenmesi

```
library(moments)

# Çarpıklık hesaplama
calculate_skewness <- function(df) {
  # Sadece numerik kolonları seç
  numeric_cols <- df[, sapply(df, is.numeric)]

  # Çarpıklık hesapla
  skewness_values <- sapply(numeric_cols, skewness)

  return(skewness_values)
}

# Çarpıklık değerlerini hesapla
skewness_results <- calculate_skewness(data)

# Sonuçları yazdır
print(skewness_results)
```

- **Yüksek çarpıklık:**
  - **Perceptions of corruption** (1.634): Yüksek pozitif çarpıklık.
  - **Social support** (-1.124): Yüksek negatif çarpıklık.

- **Orta seviye çarpıklık:**
  - **Generosity** (0.739): Pozitif çarpıklık.
  - **Freedom to make life choices** (-0.679): Negatif çarpıklık.
  - **Healthy life expectancy** (-0.608): Negatif çarpıklık.
- **Düşük çarpıklık:**
  - **GDP per capita** (-0.381): Hafif negatif çarpıklık.
  - **Score** (0.011): Simetriğe oldukça yakın bir dağılım.

1.2 Box-cox Dönüşümü

```
library(car)
# PowerTransform ile optimal lambda değerlerini bul
boxcox_transformed <- powerTransform(data[, c("Social.support",
                                              "Perceptions.of.corruption",
                                              "Healthy.life.expectancy",
                                              "Freedom.to.make.life.choices",
                                              "Generosity")] + 1)

# Belirlenen lambda değerleriyle dönüşümü uygula
data_transformed <- data
data_transformed[, c("Social.support",
                     "Perceptions.of.corruption",
                     "Healthy.life.expectancy",
                     "Freedom.to.make.life.choices",
                     "Generosity")] <- bcPower(
                       data[, c("Social.support",
                                "Perceptions.of.corruption",
                                "Healthy.life.expectancy",
                                "Freedom.to.make.life.choices",
                                "Generosity")] + 1,  # +1 ofset eklenmiş veriyi kullan
                       coef(boxcox_transformed)  # Optimal lambda değerlerini kullan
                     )

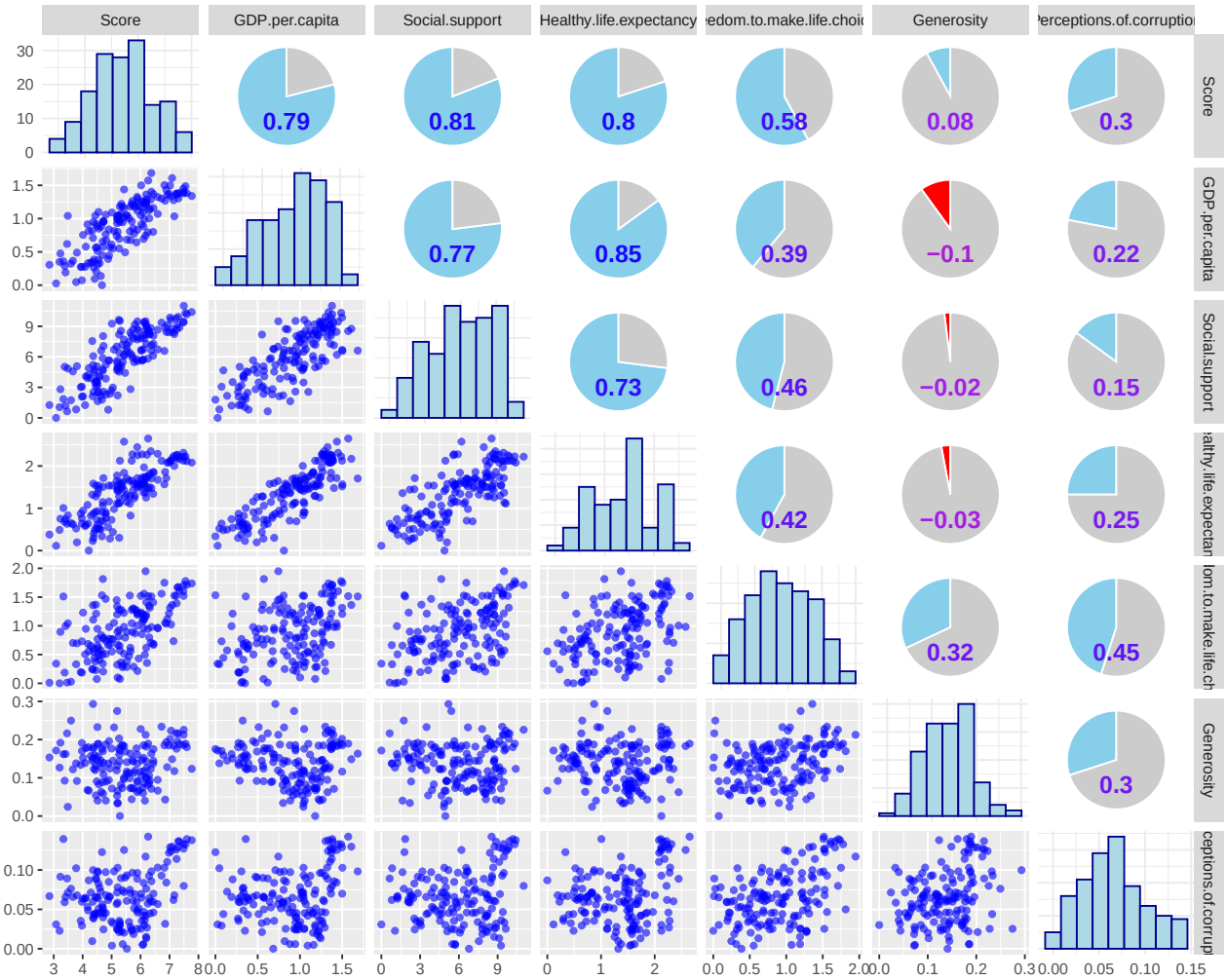
calculate_skewness(data_transformed)
```

Box-Cox dönüşümü sonrası skewness değerlerinde belirgin bir azalma görülmüştür:

- **Score**, **Generosity**, **Freedom to make life choices** ve **Healthy life expectancy** gibi değişkenler simetriğe oldukça yakın bir dağılıma ulaşmıştır.
- Başlangıçta yüksek negatif çarpıklığa sahip olan **Social support** (-1.12 → -0.28) ve yüksek pozitif çarpıklık gösteren **Perceptions of corruption** (1.63 → 0.38), daha dengeli bir yapıya kavuşmuştur.
- **GDP per capita** gibi hafif negatif çarpıklığa sahip değişkenler ise simetriğe daha da yaklaşmıştır (-0.38 → -0.28).

Bu sonuçlar, dönüşümün değişkenlerin dağılımlarını normalize etmede etkili olduğunu göstermektedir.

1.3 Veri Görselleştirmesi



Görsel, bağımsız değişkenlerin hem kendi aralarındaki ilişkileri hem de hedef değişken olan **Score**

ile olan korelasyonlarını göstermektedir. **GDP per capita**, **Social support** ve **Healthy life expectancy** arasında yüksek pozitif korelasyonlar (0.77-0.85) bulunmakta ve bu değişkenler aynı zamanda **Score** ile güçlü bir ilişki sergilemektedir (sırasıyla 0.79, 0.81 ve 0.80). Bu durum, yaşam memnuniyetinin gelir, sosyal destek ve sağlık faktörlerinden önemli ölçüde etkilendiğini göstermektedir. **Freedom to make life choices**, hem diğer değişkenlerle (örneğin **Social support** ile 0.73) hem de **Score** ile (0.58) orta düzeyde bir ilişki göstererek özgürlük algısının yaşam memnuniyetine dolaylı katkısını işaret etmektedir. Öte yandan, **Generosity** ve **Perceptions of corruption**, diğer bağımsız değişkenlerle düşük korelasyona sahip olup, **Score** ile olan ilişkileri de zayıftır (0.08 ve 0.3). Bu, bu değişkenlerin yaşam memnuniyeti üzerindeki etkilerinin sınırlı olduğunu göstermektedir.

1.4 Verinin Eğitim - Test Şeklinde Bölünmesi

Veri Eğitim-Test olarak %70 oranında bölünmüştür.

```r
# Set random seed for reproducibility
set.seed(325)

# Veri satır sayısını al
n <- nrow(data)

# %70'lik train indekslerini rastgele seç
train_indices <- sample(seq_len(n), size = 0.7 * n)

# Train ve test veri setlerini oluştur
train_data <- data[train_indices, ]
test_data <- data[-train_indices,]
```

1.5 Box-Cox Lambda Değerlerinin Eğitim Verisi Üzerinden Bulunması

Box-Cox dönüşümü sırasında herhangi bir overfitting oluşmaması adına, lambda değerleri eğitim verisi üzerinden elde edilmiştir. Elde edilen lambda değeri ile eğitim veri üzerinde dönüşümler yapılmıştır.

```r
# PowerTransform ile optimal lambda değerlerini bul (sadece train verisi üzerinde)
boxcox_transformed <- powerTransform(train_data[, c("Social.support",
                                                    "Perceptions.of.corruption",
                                                    "Healthy.life.expectancy",
                                                    "Freedom.to.make.life.choices",
                                                    "Generosity")] + 1)
```

```r
# Lambda değerlerini kontrol et
boxcox_transformed

# Belirlenen lambda değerleriyle dönüşümü uygula
train_data_transformed <- train_data
test_data_transformed <- test_data

# Train verisine dönüşüm uygula
train_data_transformed[, c("Social.support",
                           "Perceptions.of.corruption",
                           "Healthy.life.expectancy",
                           "Freedom.to.make.life.choices",
                           "Generosity")] <- bcPower(
                             train_data[, c("Social.support",
                                            "Perceptions.of.corruption",
                                            "Healthy.life.expectancy",
                                            "Freedom.to.make.life.choices",
                                            "Generosity")] + 1,  # +1 ofset eklenmiş veriyi kullan
                             coef(boxcox_transformed)  # Optimal lambda değerlerini kullan
                           )
```

1.6 Box-Cox Dönüşümünün Test Verisine Uygulanması

Eğitim verisi ile elde edilen lambda değerleri kullanılarak test verisinde de aynı şekilde box-cox dönüşümü yapılmıştır.

```r
# Test verisine dönüşüm uygula (Train'den bulunan lambda değerleriyle)
test_data_transformed[, c("Social.support",
                          "Perceptions.of.corruption",
                          "Healthy.life.expectancy",
                          "Freedom.to.make.life.choices",
                          "Generosity")] <- bcPower(
                            test_data[, c("Social.support",
                                          "Perceptions.of.corruption",
                                          "Healthy.life.expectancy",
                                          "Freedom.to.make.life.choices",
                                          "Generosity")] + 1,  # +1 ofset eklenmiş veriyi kullan
```

```
                              coef(boxcox_transformed)  # Optimal lambda değerlerini kullan
                            )
```

2 Regresyon Modellerinin Geliştirilmesi

2.1 Regresyon Metriklerinin Tanımlanması

Regresyon metrikleri olarak MAE, MAPE, MSE, RMSE ve R^2 seçilmiştir.

2.2 Lineer Regresyon

```
library(olsrr)
#str(train_data)
model1 <- lm(Score ~ ., data = train_data_transformed)
a=ols_step_all_possible(model1)
#a$result
#View(a$result)
#plot(a)
#a$result$predictors[57]

model2 <- lm(Score ~ ., data = train_data_transformed[c("Score","GDP.per.capita",
                                                        "Social.support",
                                                        "Healthy.life.expectancy",
                                                        "Freedom.to.make.life.choices",
                                                        "Perceptions.of.corruption")])
summary(model2)
```

```
##
## Call:
## lm(formula = Score ~ ., data = train_data_transformed[c("Score",
##     "GDP.per.capita", "Social.support", "Healthy.life.expectancy",
##     "Freedom.to.make.life.choices", "Perceptions.of.corruption")])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.23158 -0.32180  0.04779  0.31220  1.17834
```

```
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   2.81553    0.14291  19.701  < 2e-16 ***
## GDP.per.capita                0.52357    0.23374   2.240 0.027242 *
## Social.support                0.11585    0.02841   4.078 8.98e-05 ***
## Healthy.life.expectancy       0.54061    0.14202   3.807 0.000239 ***
## Freedom.to.make.life.choices  0.53504    0.12456   4.295 3.96e-05 ***
## Perceptions.of.corruption     3.32079    1.60042   2.075 0.040484 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4732 on 103 degrees of freedom
## Multiple R-squared:  0.8047, Adjusted R-squared:  0.7953
## F-statistic: 84.89 on 5 and 103 DF,  p-value: < 2.2e-16
```

```
shapiro.test(residuals(model2))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(model2)
## W = 0.99358, p-value = 0.8951
```

```
library(Hmisc)
library(car)
library(lmtest)

bptest(model2)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  model2
## BP = 5.8538, df = 5, p-value = 0.3207
```

```
vif(model2)
```

```
##               GDP.per.capita                Social.support
##                     4.107271                      2.579311
##      Healthy.life.expectancy Freedom.to.make.life.choices
```

```
##                      3.646374                      1.527308
##    Perceptions.of.corruption
##                      1.225365
```

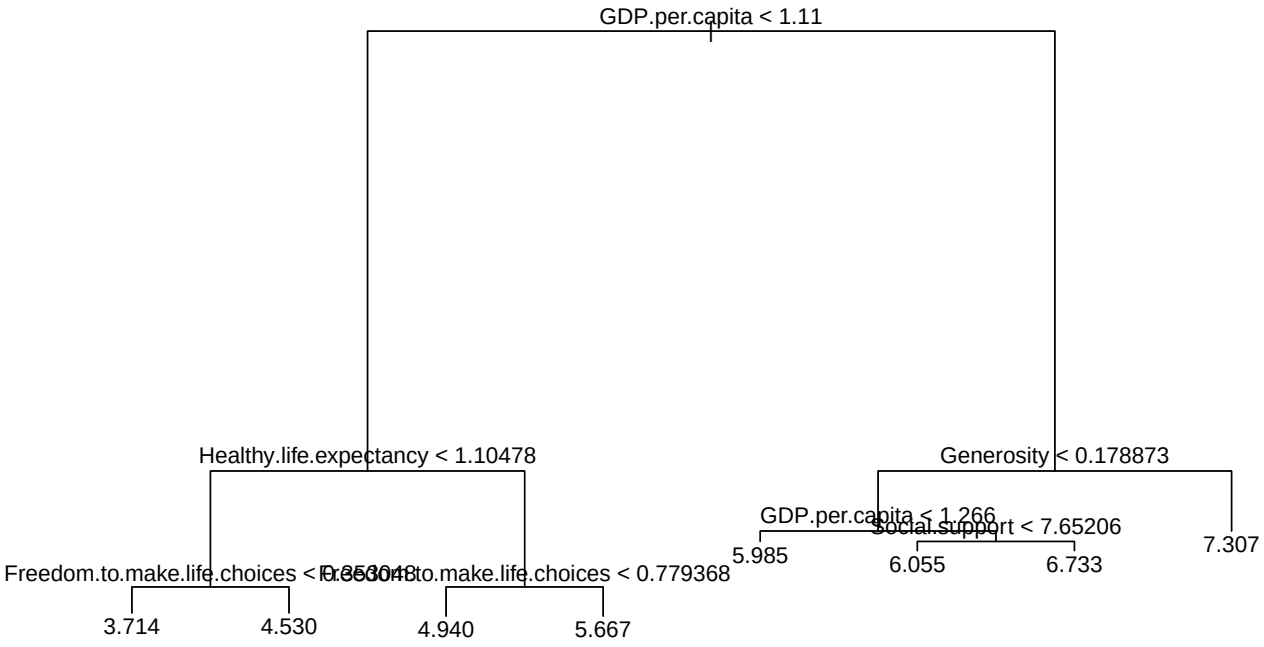
Lineer regresyon modeli oluşturulurken, ilk aşamada tüm değişkenler modele dahil edilerek **Model 1** oluşturulmuştur. Daha sonra **all possible regressions** yöntemiyle değişkenlerin tüm alt kümeleri değerlendirilmiş ve değişken sayısı ile **Cp** değeri arasındaki farkın en küçük olduğu **57. model** seçilmiştir. Bu modelde, orijinal veri setinden yalnızca **Generosity** değişkeni çıkarılmıştır.

Kurulan modelin şu özellikleri bulunmaktadır: - Modeldeki tüm parametreler **istatistiksel olarak anlamlı** bulunmuştur. - Modelin artıklarının **normal dağılıma uygunluğu**, **Shapiro-Wilk testi** ile doğrulanmıştır. - **Breusch-Pagan testi** sonuçlarına göre, homoskedastisite varsayımının ihlal edildiğine dair **yeterli kanıt bulunamamıştır**. - **Variance Inflation Factor (VIF)** değerleri incelendiğinde, hiçbir değişkenin 5'in üzerinde bir değere sahip olmadığı görülmüştür. Bu da modelde **multikolineerlik probleminin olmadığını** göstermektedir.

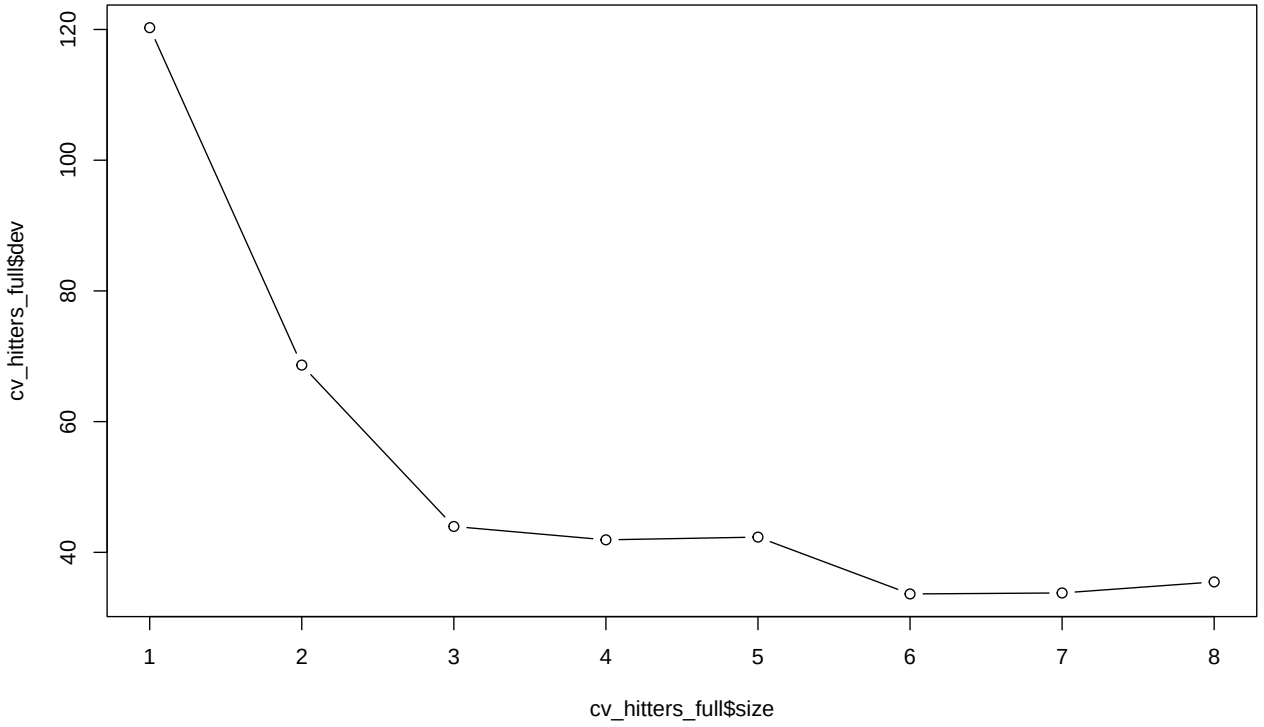
Bu sonuçlar, seçilen modelin istatistiksel açıdan **sağlam ve güvenilir** olduğunu göstermektedir.

2.3 Regresyon Ağacı

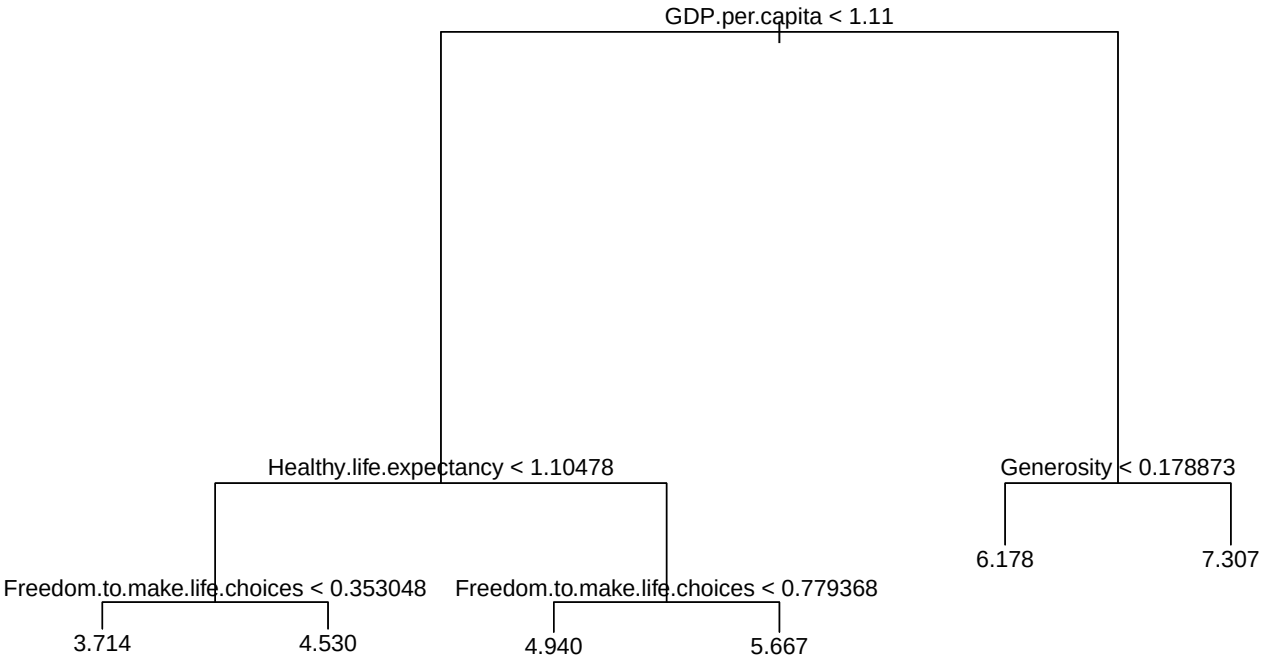
```
library(tree)
set.seed(325)
tree_hitters_full <-tree(Score~., data=train_data_transformed)
plot(tree_hitters_full)
text(tree_hitters_full)
```




```
cv_hitters_full <- cv.tree(tree_hitters_full)
plot(cv_hitters_full$size,cv_hitters_full$dev,type='b')
```




```
prune_hitters_full=prune.tree(tree_hitters_full,best=6)
plot(prune_hitters_full)
text(prune_hitters_full,pretty=0)
```



Regresyon ağacı sonucunda, Generosity değişkeninin Lineer Regresyon modelinde çıkarılmasına rağmen, burada en önemli featurelardan birisi olması dikkat çekmiştir. 2. grafikte ağaç budama için 6 dalın en düşük değeri aldığı gözükmektedir. Bu sebeple, regresyon ağacında hem budama yapılmış hem budama yapılmamış model dikkate alınmıştır.

2.4 Bagging

```
set.seed(325)
library(randomForest)
bag<-randomForest(Score~.,data = train_data_transformed,mtry=6,importance=TRUE)
```

Bagging, yapılırken veride 6 feature bulunduğu için mtry değeri 6 olarak girilmiştir.

2.5 Random Forest

```
set.seed(325)
rf<-randomForest(Score~.,data = train_data_transformed,mtry=2,importance=TRUE)
```

Random Forest, yapılırken veride 6 feature bulunduğu için mtry değeri 6/3=2 'den dolayı 2 olarak girilmiştir.

2.6 Genel Değerlendirme

```
list(
  linear_regression            = lm_results_df,
  unpruned_tree = unpruned_tree_results_df,
  pruned_tree   = pruned_tree_results_df,
  random_forest             = rf_results_df
)
```

```
## $linear_regression
##    Metric      Train       Test
## 1  lm_MAE 0.36831391 0.46073860
## 2 lm_MAPE 0.07321795 0.09657854
## 3  lm_MSE 0.21160394 0.37351640
## 4 lm_RMSE 0.46000428 0.61115988
## 5   lm_R2 0.80472993 0.76241050
##
## $unpruned_tree
##               Metric      Train      Test
## 1  unpruned_tree_MAE 0.31807151 0.5642077
## 2 unpruned_tree_MAPE 0.06331139 0.1136418
## 3  unpruned_tree_MSE 0.15858461 0.4444843
## 4 unpruned_tree_RMSE 0.39822683 0.6666966
## 5   unpruned_tree_R2 0.85365666 0.7172686
##
## $pruned_tree
##             Metric      Train      Test
## 1  pruned_tree_MAE 0.33527632 0.6296219
## 2 pruned_tree_MAPE 0.06596072 0.1229180
## 3  pruned_tree_MSE 0.18491162 0.5614284
## 4 pruned_tree_RMSE 0.43001351 0.7492853
## 5   pruned_tree_R2 0.82936185 0.6428818
##
## $random_forest
##    Metric      Train      Test
## 1  rf_MAE 0.16323613 0.4725240
## 2 rf_MAPE 0.03285738 0.1003085
## 3  rf_MSE 0.04497114 0.3533719
```

```
## 4 rf_RMSE 0.21206401 0.5944509
## 5   rf_R2 0.95850021 0.7752242
```

Model performanslarının değerlendirilmesinde kullanılan metrikler (MAE, MAPE, MSE, RMSE ve R^2) üzerinden bir analiz yapılmış ve her modelin eğitim ve test veri setlerindeki sonuçları incelenmiştir. Değerlendirme sonucunda **Lineer Regresyon (lm)** modeli en güvenilir model olarak seçilmiştir.

2.6.1 Değerlendirme Kriterleri

1. **Tahmin Hataları (RMSE, MAE, MAPE):** Lineer Regresyon modeli, özellikle **RMSE (test: 0.6111)** ve **MAE (test: 0.4607)** değerleriyle test veri setindeki hataları minimum düzeyde tutmayı başarmıştır. Ayrıca **MAPE (test: 0.0966)** değeri, diğer modellere kıyasla daha düşük bir hata oranına işaret etmektedir. Bu, modelin test verisindeki tahmin doğruluğunun yüksek olduğunu göstermektedir.

2. **Eğitim ve Test Metriklerinin Birbirine Yakınlığı:** Lineer Regresyon modelinde eğitim ve test seti arasındaki metrik farklarının oldukça düşük olduğu gözlemlenmiştir. Örneğin, **R^2** değerleri eğitim setinde **0.8047**, test setinde ise **0.7624** olarak oldukça yakın değerlerdedir. Bu durum, modelin genelleme kabiliyetinin yüksek olduğunu ve overfitting (aşırı öğrenme) probleminin bulunmadığını göstermektedir.

3. **Model Basitliği ve Güvenilirlik:** Lineer Regresyon, basit yapısıyla diğer modellere kıyasla daha açıklayıcı ve yorumlanabilir bir çözüm sunmaktadır. Parametrelerin anlamlı olması, modelin sağlam bir istatistiksel temele dayandığını desteklemektedir.

2.6.2 Diğer Modellerle Karşılaştırma

- **Random Forest:** En düşük test seti hatalarından bazılarına sahip olmasına rağmen, eğitim ve test metrikleri arasındaki farklar Lineer Regresyon'a göre daha yüksektir. Bu durum, Random Forest modelinin genelleme kabiliyetinin Lineer Regresyon kadar güvenilir olmadığını göstermektedir.
- **Unpruned ve Pruned Decision Tree:** Karar ağacı modelleri, test setindeki hatalarının yüksekliği ve düşük genelleme kabiliyetleri sebebiyle Lineer Regresyon kadar güvenilir bulunmamıştır.

2.6.3 Sonuç

Eğitim ve test setindeki metriklerin birbirine yakın olması, düşük tahmin hataları (MAE, RMSE, MAPE) ve modelin genelleme kabiliyeti dikkate alındığında, **Lineer Regresyon modeli** bu veri seti

için en güvenilir ve tercih edilebilir model olarak değerlendirilmiştir. Lineer regresyonda, Genosisity değişkeni kullanılmadığı için, bu parametre'nin model üzerinde bir etkisi olmadığına karar verilmiştir.

3 Sınıflandırma Modelleri

3.1 Sınıflandırma İçin Eşik Değerlerinin Belirlenmesi

Verinin dengeli ve eşit iki parçaya bölünebilmesi adına, medyan değeri eşik değer olarak belirlenmiştir. Medyan değerinden büyük olanlar 1, küçük ve eşit olanlar 0 şeklinde kodlanmıştır. Modelin doğru geliştirilebilmesi adına, sonrasında hedef değişken factor olarak tanımlanmıştır.

```
threshold <- median(c(train_data_transformed$Score,test_data_transformed$Score))
train_data_transformed$Score <- ifelse(threshold < train_data_transformed$Score,1,0)
test_data_transformed$Score <- ifelse(threshold < test_data_transformed$Score,1,0)
table(c(train_data_transformed$Score,test_data_transformed$Score))
```

```
##
##  0  1
## 78 78
```

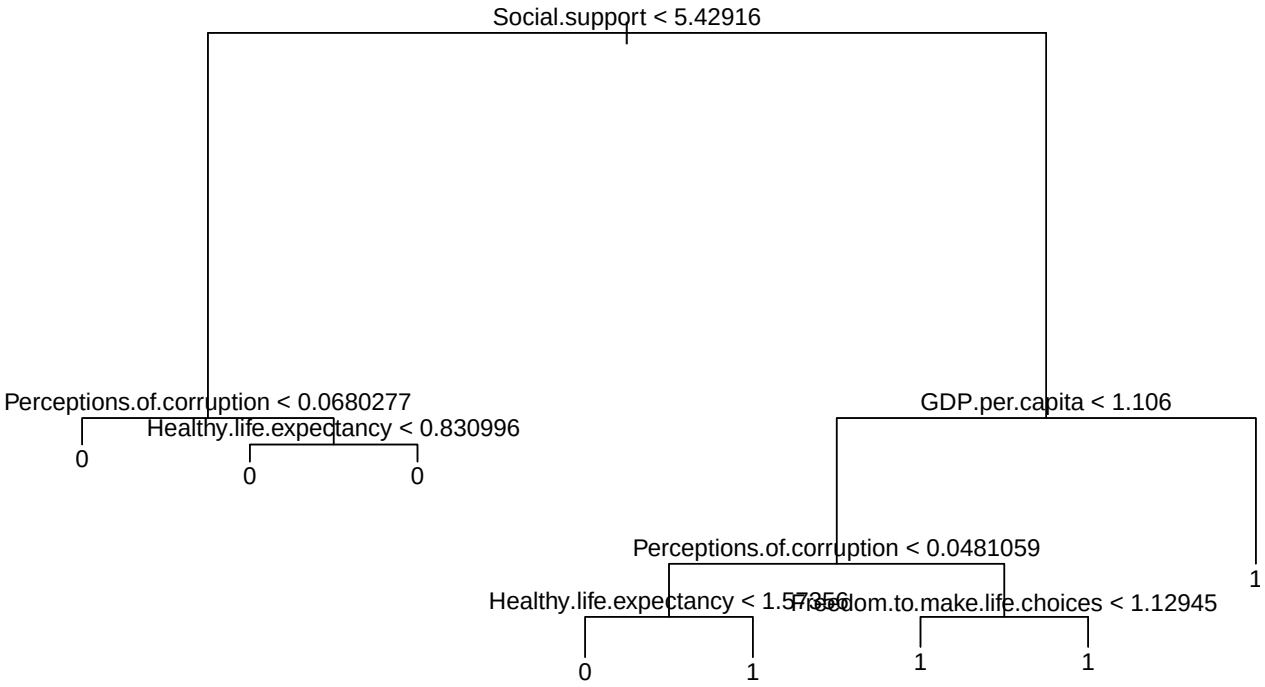
```
train_data_transformed$Score <- as.factor(train_data_transformed$Score)
test_data_transformed$Score <- as.factor(test_data_transformed$Score)
```

3.2 Sınıflandırma Metriklerinin Tanımlanması

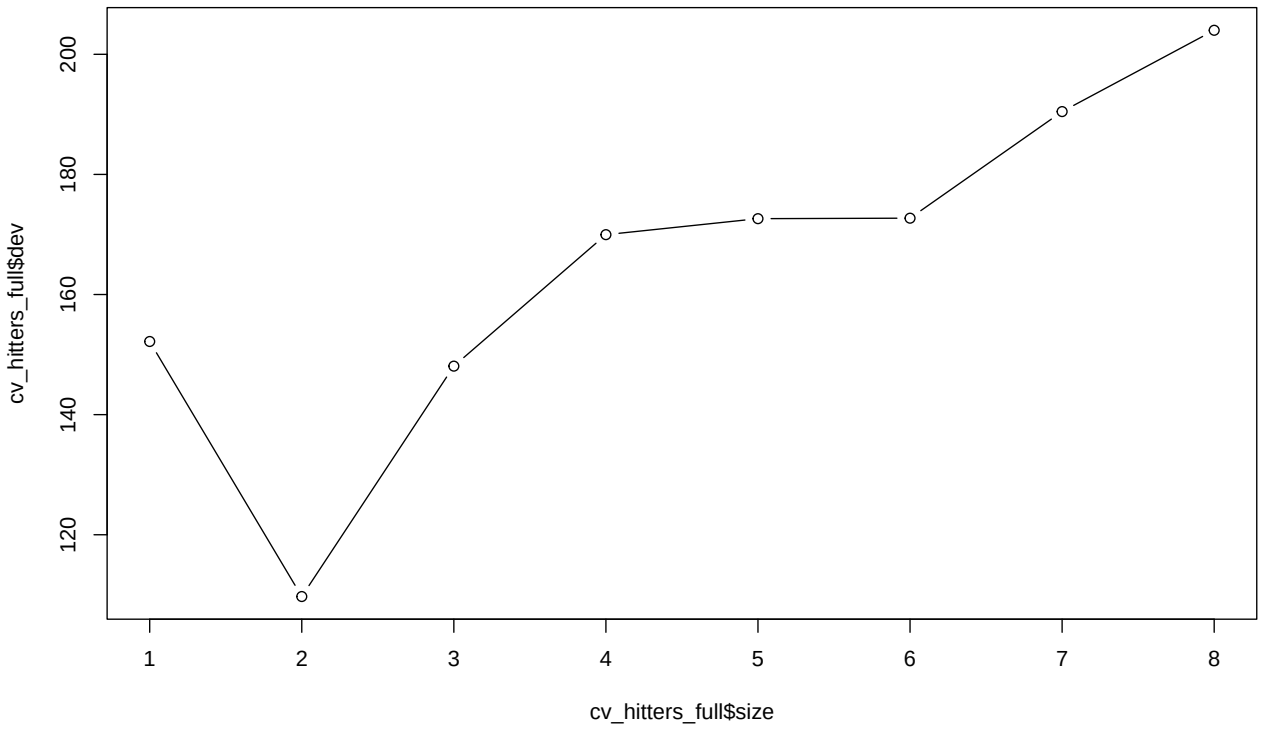
Sınıflandırma metrikleri olarak f1 score, accuracy, precision, recall, sensitivity ve specifity seçilmiştir.

3.3 Sınıflandırma Ağacı

```
set.seed(325)
tree_hitters_full <-tree(Score~., data=train_data_transformed)
plot(tree_hitters_full)
text(tree_hitters_full)
```

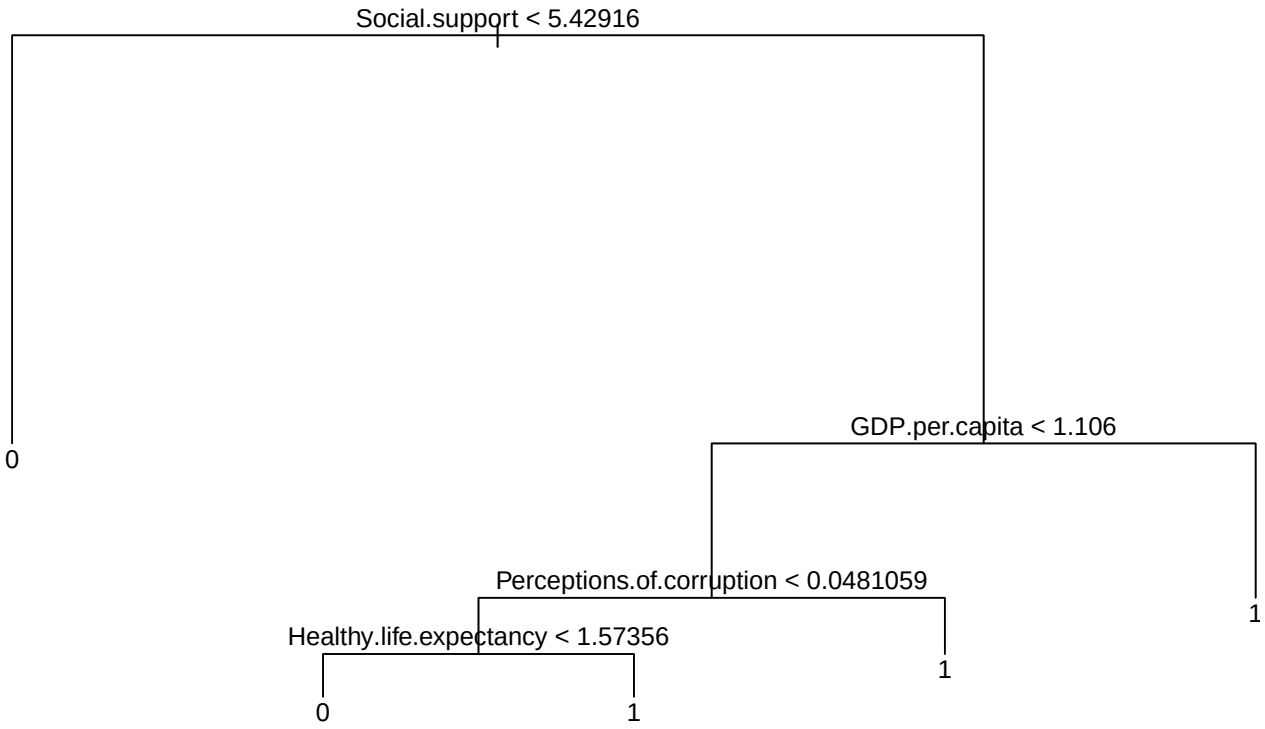



```
cv_hitters_full <- cv.tree(tree_hitters_full)
plot(cv_hitters_full$size,cv_hitters_full$dev,type='b')
```




```
prune_hitters_full=prune.tree(tree_hitters_full,best=5)
plot(prune_hitters_full)
```

```
text(prune_hitters_full,pretty=0)
```



Prune edilmemiş ağaçta, her iki dalının da aynı değer verdiği bir sürü dal fark edilmiştir. Grafik, en ideal dal sayısını 2 olarak belirlemiş olsa da, test verisindeki metrikler yeterli bulunmadığı için 3,4,5,6 alternatifi denenmiştir ve en iyi sonuçlar 6'da elde edildiği için 6 dal sayısı olarak belirlenmiştir. 6 dala bakıldığında da iki tarafın aynı değeri tahmin ettiği bir tahmin değeri elde edilmemiştir.

3.4 Bagging

```
set.seed(325)
bag<-randomForest(Score~.,data = train_data_transformed,mtry=6,importance=TRUE)
```

Bagging, yapılırken veride 6 feature bulunduğu için mtry değeri 6 olarak girilmiştir.

3.5 Random Forest

```
set.seed(325)
rf<-randomForest(Score~.,data = train_data_transformed,mtry=2,importance=TRUE)
```

Random Forest, yapılırken veride 6 feature bulunduğu için mtry değeri 6/3=2 'den dolayı 2 olarak girilmiştir.

3.6 Lojistik Regresyon

```
library(MASS)

model1_logistic <- glm(Score ~ .,
                       data = train_data_transformed,
                       family = binomial)
model2_logistic=stepAIC(model1_logistic, direction = "both", trace = F)
summary(model2_logistic)
```

```
##
## Call:
## glm(formula = Score ~ Social.support + Healthy.life.expectancy +
##     Freedom.to.make.life.choices + Perceptions.of.corruption,
##     family = binomial, data = train_data_transformed)
##
## Coefficients:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -13.6240     3.1788  -4.286 1.82e-05 ***
## Social.support                 0.5943     0.1991   2.985 0.002840 **
## Healthy.life.expectancy        4.8497     1.4470   3.352 0.000803 ***
## Freedom.to.make.life.choices   1.8601     0.9136   2.036 0.041753 *
## Perceptions.of.corruption     30.7616    14.7759   2.082 0.037353 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 151.097  on 108  degrees of freedom
## Residual deviance:  52.799  on 104  degrees of freedom
## AIC: 62.799
##
## Number of Fisher Scoring iterations: 7
```

```r
vif(model2_logistic)
```

```
##               Social.support      Healthy.life.expectancy
##                     1.172107                     1.342037
## Freedom.to.make.life.choices    Perceptions.of.corruption
##                     1.153564                     1.387192
```

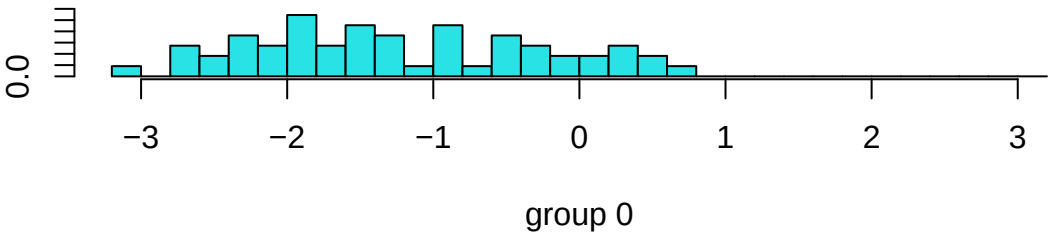
1. **Model Seçimi:**
   - `stepAIC` ile `model1_logistic` üzerinde AIC bazlı ileri ve geri değişken seçimi yapılmış, optimize edilmiş model olarak `model2_logistic` elde edilmiştir.
2. **Seçilen Değişkenler:**
   - **Social.support**, **Healthy.life.expectancy**, **Freedom.to.make.life.choices**, **Perceptions.of.corruption** seçilmiştir. Tüm değişkenler anlamlıdır ($\mathbf{p < 0.05}$).
3. **Model Performansı:**
   - **AIC (62.799):** Düşük AIC değeri modelin uygun olduğunu gösterir.
   - **Residual Deviance (52.799):** Model, bağımsız değişkenlerle hedef değişkeni iyi açıklamaktadır.
   - VIF değerleri düşük olup çoklu doğrusallık sorunu bulunmamaktadır.

3.7 LDA (Linear Discriminant Analysis)

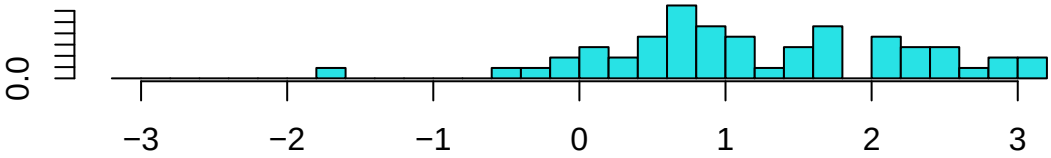
```r
#LDA
set.seed(325)

model_lda<-lda(Score ~ .,
    data = train_data_transformed[,c("Social.support", "Healthy.life.expectancy",
                                     "Freedom.to.make.life.choices",
                                     "Perceptions.of.corruption","Score")])
#model_lda

lda_train_pred <- predict(model_lda, train_data_transformed[,c("Social.support", "Healthy.life.exp
                                                               "Freedom.to.make.life.choices",
                                                               "Perceptions.of.corruption","Score"
plot(model_lda)
```

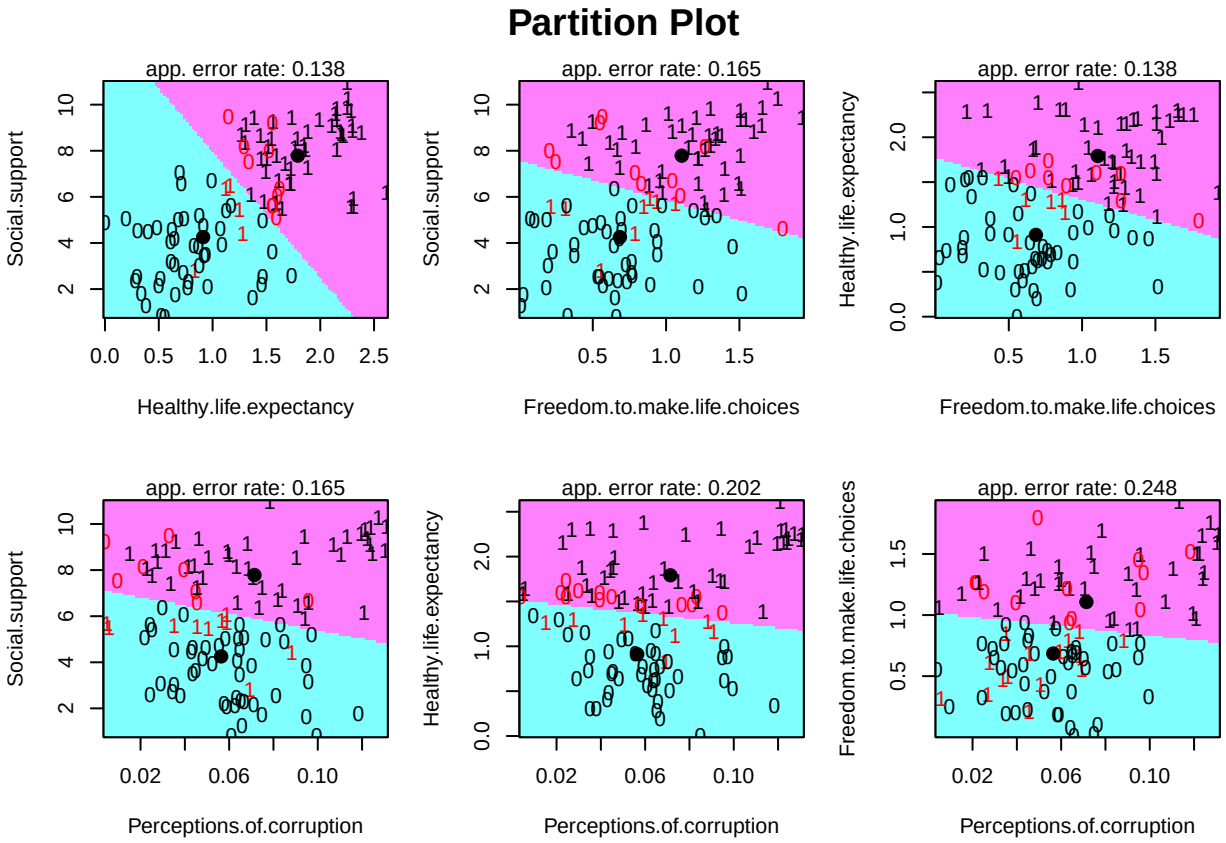


group 0



group 1

```
library(klaR)
partimat(Score~., data=train_data_transformed[,c("Social.support", "Healthy.life.expectancy",
                                                 "Freedom.to.make.life.choices","Perceptions.of.co
         method="lda")
```



LDA modeli, yöntemin çoklu doğrusallık hassasiyeti nedeniyle, lojistik regresyonda AIC ile seçilen değişkenlerle oluşturulmuştur (**Social.support**, **Healthy.life.expectancy**, **Freedom.to.make.life.choices**, **Perceptions.of.corruption**). Bu seçim, model stabilitesini ve değişkenlerin sınıfları ayırmadaki etkinliğini artırmıştır.

1. **Prior Olasılıklar:** Sınıflar dengelidir (%50.46 ve %49.54).
2. **Grup Ortalamaları:** Sınıf 1'in değişken ortalamaları daha yüksektir, özellikle **Social.support** ve **Healthy.life.expectancy** belirgin fark yaratır.
3. **Ayrım Katsayıları:** Veri standartlaştırılmadığı için katsayı büyüklükleri doğrudan etkili değişkenleri göstermez.

3.7.1 Histogramlar:

- Sınıf 1 gözlemleri pozitif, sınıf 0 gözlemleri negatif değerlere yoğunlaşmıştır. Ayrım genel olarak net, ancak örtüşmeler vardır.

3.7.2 Partition Plot:

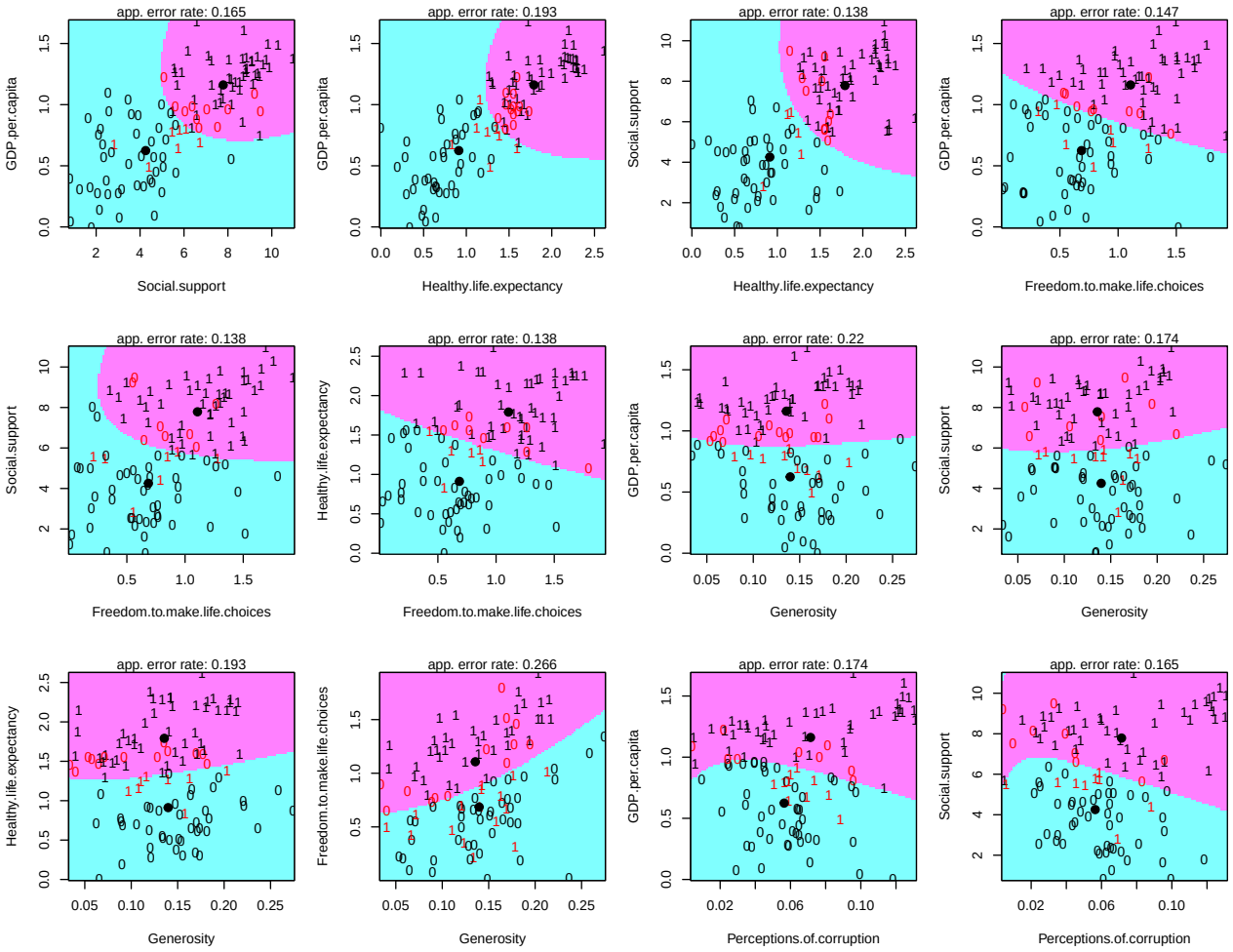
- **Healthy.life.expectancy** ve **Social.support** en düşük hata oranını (0.138) vermektedir. Diğer değişken kombinasyonlarında hata oranları daha yüksektir.

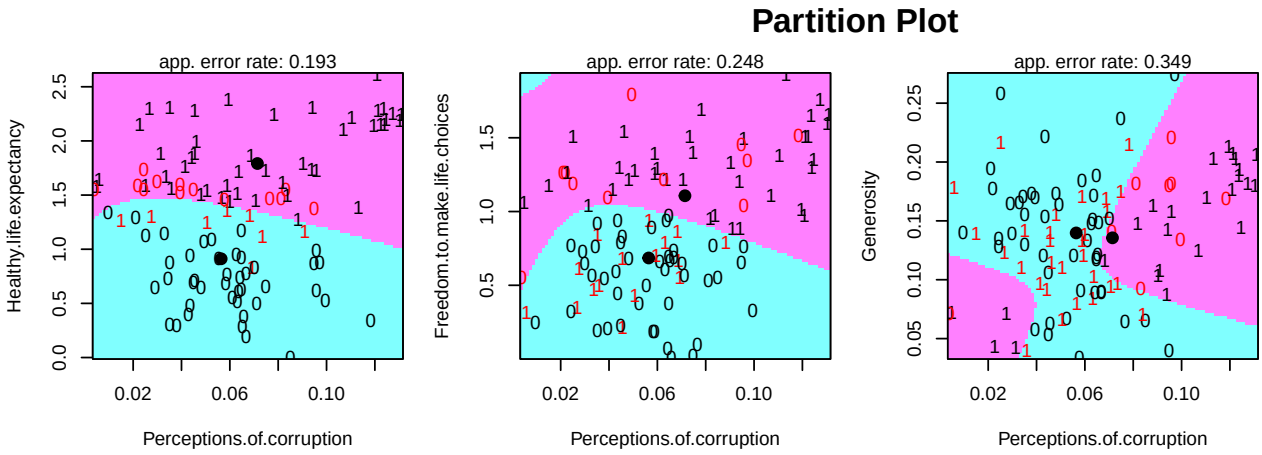
3.8 QDA (Quadratic Discriminant Analysis)

```
#QDA
model_qda<-qda(Score ~ .,
               data = train_data_transformed)

qda_train_pred <- predict(model_qda, train_data_transformed)$class

partimat(Score~ ., data=train_data_transformed,method="qda")
```





QDA modeli, çoklu doğrusallığa LDA'ya göre daha dayanıklı olduğu için tüm değişkenler kullanılarak oluşturulmuş ve sınıflar arasındaki doğrusal olmayan ayrımları etkili bir şekilde modellemiştir.

1. **Genel Ayrım:** QDA, esnek sınırlarla sınıflar arasında net bir ayrım sağlamış, ancak bazı değişken kombinasyonlarında örtüşme gözlenmiştir.

2. **Hata Oranları: Healthy.life.expectancy** ve **Social.support** düşük hata oranı (0.138-0.193) ile en iyi ayrımı sağlarken, **Generosity** ve **Perceptions.of.corruption** kombinasyonları daha yüksek hata oranı göstermiştir (0.266-0.349).

Model, doğrusal olmayan sınırlarla karmaşık ayrımları etkili şekilde yakalamıştır.

3.9 ROC - AUC

3.9.1 ROC Nedir?

ROC (Receiver Operating Characteristic) eğrisi, bir modelin sınıflandırma performansını değerlendirmek için kullanılır. Bu eğri, farklı eşik değerlerinde modelin **True Positive Rate** (Sensitivity) ve **False Positive Rate** (1 - Specificity) arasındaki ilişkiyi gösterir.

3.9.2 AUC Nedir?

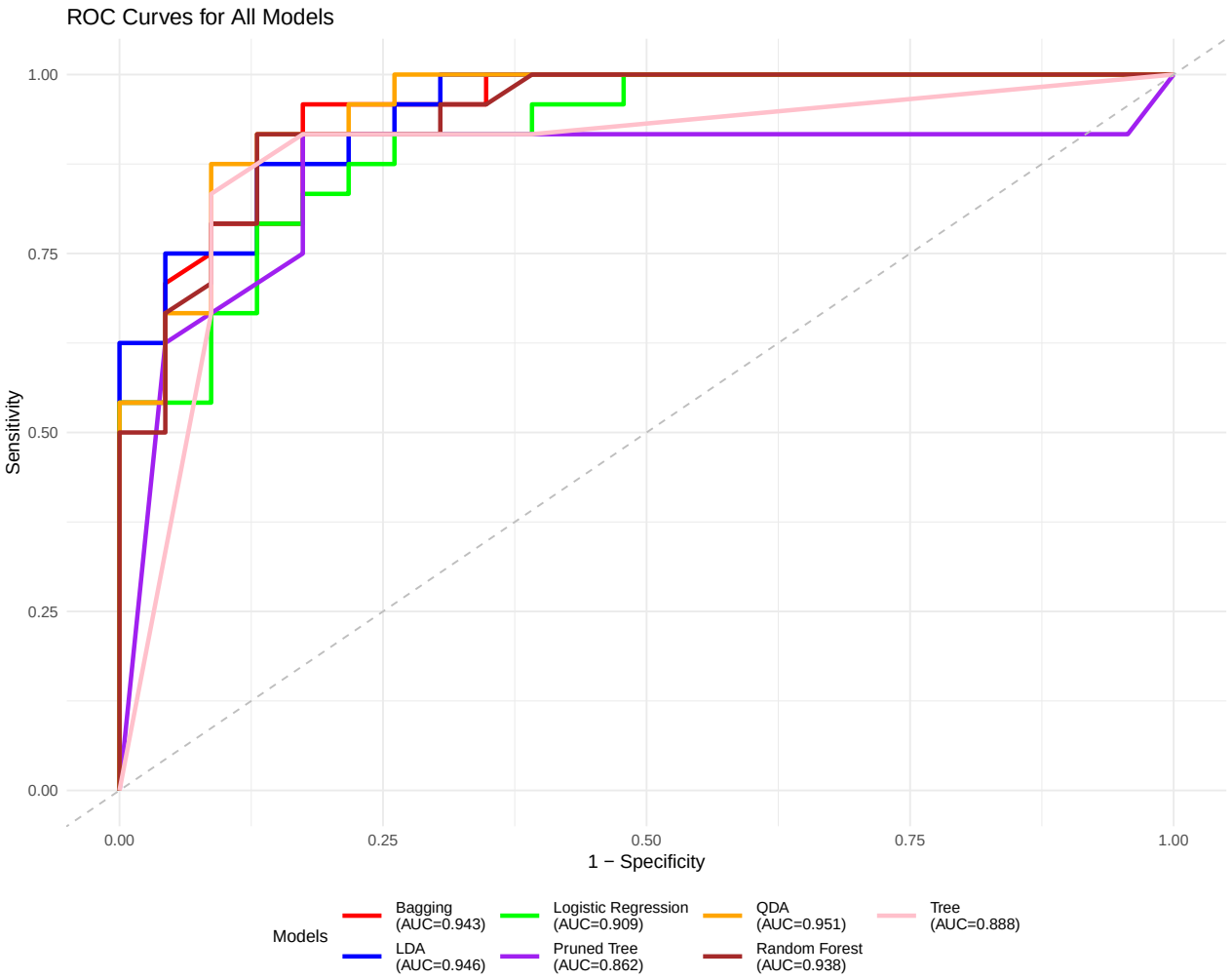
AUC (Area Under Curve), ROC eğrisinin altında kalan alanı ifade eder ve modelin genel performansını ölçer. AUC değeri 1'e ne kadar yakınsa, modelin performansı o kadar iyidir.

3.9.3 Bu Çalışmada Yapılanlar

1. **Model Tahminleri:**

- Modeller test veri setine uygulanmış ve `predict()` fonksiyonu ile tahmin olasılıkları hesaplanmıştır.

2. **ROC Hesaplama:**
   - Tahmin edilen olasılıklar ve gerçek sınıf etiketleri, `pROC` kütüphanesinin `roc()` fonksiyonu ile ROC eğrisi oluşturmak için kullanılmıştır.
3. **AUC Hesaplama:**
   - ROC eğrisinin altında kalan alan, `auc()` fonksiyonu ile hesaplanmıştır.



1. **En Yüksek Performans:** - **QDA (AUC=0.951)** ve **LDA (AUC=0.946)** modelleri en iyi sonuçları vermiştir ve yüksek doğruluk sergilemektedir.

2. **Ensemble Modeller:**
   - **Bagging (AUC=0.943)** ve **Random Forest (AUC=0.938)**, güçlü performans göstermiştir ve dengeli sonuçlar sunmuştur.
3. **Orta Performans:**
   - **Logistic Regression (AUC=0.909)** ve **Unpruned Tree (AUC=0.888)** orta düzeyde

başarılıdır.

4. **Düşük Performans:**
   - **Pruned Tree (AUC=0.862)** diğer modellere kıyasla düşük performans sergilemiş, budama işlemi modelin genelleme yeteneğini azaltmıştır.

3.10 Sınıflandırma Sonuçlarının Değerlendirilmesi

```
## $`Unpruned Classification Tree`
##                       Metric     Train      Test
## 1          unpruned_tree_F1 0.9369369 0.8800000
## 2    unpruned_tree_Accuracy 0.9357798 0.8723404
## 3   unpruned_tree_Precision 0.9629630 0.9166667
## 4      unpruned_tree_Recall 0.9122807 0.8461538
## 5 unpruned_tree_Sensitivity 0.9122807 0.8461538
## 6 unpruned_tree_Specificity 0.9615385 0.9047619
##
## $`Pruned Classification Tree`
##                     Metric     Train      Test
## 1          pruned_tree_F1 0.9369369 0.8800000
## 2    pruned_tree_Accuracy 0.9357798 0.8723404
## 3   pruned_tree_Precision 0.9629630 0.9166667
## 4      pruned_tree_Recall 0.9122807 0.8461538
## 5 pruned_tree_Sensitivity 0.9122807 0.8461538
## 6 pruned_tree_Specificity 0.9615385 0.9047619
##
## $`Bagging Model`
##            Metric Train      Test
## 1          bag_F1     1 0.8333333
## 2    bag_Accuracy     1 0.8297872
## 3   bag_Precision     1 0.8333333
## 4      bag_Recall     1 0.8333333
## 5 bag_Sensitivity     1 0.8333333
## 6 bag_Specificity     1 0.8260870
##
## $`Random Forest`
##           Metric Train      Test
## 1          rf_F1     1 0.8979592
```

```
## 2    rf_Accuracy     1 0.8936170
## 3   rf_Precision     1 0.9166667
## 4      rf_Recall     1 0.8800000
## 5 rf_Sensitivity     1 0.8800000
## 6 rf_Specificity     1 0.9090909
##
## $`Logistic Regression (GLM)`
##            Metric     Train      Test
## 1          glm_F1 0.8727273 0.8235294
## 2    glm_Accuracy 0.8715596 0.8085106
## 3   glm_Precision 0.8888889 0.8750000
## 4      glm_Recall 0.8571429 0.7777778
## 5 glm_Sensitivity 0.8571429 0.7777778
## 6 glm_Specificity 0.8867925 0.8500000
##
## $`Linear Discriminant Analysis (LDA)`
##            Metric     Train      Test
## 1          lda_F1 0.8828829 0.8461538
## 2    lda_Accuracy 0.8807339 0.8297872
## 3   lda_Precision 0.9074074 0.9166667
## 4      lda_Recall 0.8596491 0.7857143
## 5 lda_Sensitivity 0.8596491 0.7857143
## 6 lda_Specificity 0.9038462 0.8947368
##
## $`Quadratic Discriminant Analysis (QDA)`
##            Metric     Train      Test
## 1          qda_F1 0.9158879 0.8979592
## 2    qda_Accuracy 0.9174312 0.8936170
## 3   qda_Precision 0.9074074 0.9166667
## 4      qda_Recall 0.9245283 0.8800000
## 5 qda_Sensitivity 0.9245283 0.8800000
## 6 qda_Specificity 0.9107143 0.9090909
```

3.10.1 ROC-AUC Değerleri ve Genel Performans:

Önceki ROC eğrisi grafiği incelendiğinde, **QDA (AUC=0.951)** en yüksek AUC değerine sahip modeldir. **LDA (AUC=0.946)** ve **Bagging (AUC=0.943)** modelleri de yüksek AUC değerleriyle dikkat çekmiştir. **Random Forest (AUC=0.938)**, **Logistic Regression (AUC=0.909)** ve **Unpruned**

Tree (AUC=0.888) sırasıyla bu modelleri takip etmiştir. **Pruned Tree (AUC=0.862)** ise diğer modellere göre daha düşük bir performans göstermiştir.

3.10.2 Eğitim ve Test Performanslarının Karşılaştırılması:

1. **QDA (Quadratic Discriminant Analysis):**
   - Eğitim (F1: 0.916, Accuracy: 0.917) ve test setinde (F1: 0.898, Accuracy: 0.894) tutarlı sonuçlar vermiştir. Eğitim ve test metrikleri arasındaki farkın düşük olması, modelin genelleme yeteneğini öne çıkarır.
2. **LDA (Linear Discriminant Analysis):**
   - Eğitim (F1: 0.883, Accuracy: 0.881) ve test setinde (F1: 0.846, Accuracy: 0.830) QDA'dan sonra en güvenilir sonuçları vermiştir.
3. **Random Forest:**
   - Eğitim setinde mükemmel sonuçlar (F1: 1.0, Accuracy: 1.0) elde etmiştir, ancak test setinde performansı (F1: 0.898, Accuracy: 0.894) eğitim setine göre daha düşük kalmıştır. Bu fark, modelin aşırı öğrenme yapmış olabileceğini göstermektedir.
4. **Bagging:**
   - Eğitim setinde tüm metriklerde mükemmel sonuçlar (F1, Accuracy: 1.0) vermesine rağmen test setinde daha düşük performans göstermiştir (F1: 0.833, Accuracy: 0.830). Bu da genelleme yeteneğinin sınırlı olduğunu göstermektedir.
5. **Pruned ve Unpruned Trees:**
   - Ağaç tabanlı modeller benzer sonuçlar vermiştir. Ancak her iki model de test setinde diğer modellere göre daha düşük performans göstermiştir.
6. **Logistic Regression:**
   - Eğitim ve test setinde dengeli sonuçlar vermiştir. Ancak test setinde (F1: 0.824, Accuracy: 0.809) performansı QDA ve LDA'ya göre daha düşüktür.

3.10.3 Sonuç:

- **QDA**, eğitim ve test metrikleri arasındaki düşük farkı ve AUC skorundaki üstünlüğü (0.951) sayesinde en güvenilir model olarak seçilmiştir.
- **LDA**, benzer şekilde genelleme yeteneği yüksek olup QDA'dan sonra en güvenilir modeldir.
- **Random Forest** ve **Bagging**, eğitim setinde mükemmel sonuçlar vermiş olsa da, test seti performanslarındaki düşüş nedeniyle genelleme açısından daha az güvenilirdir.